

Technical Reference

The Map Generation Package

Complete explanation of every Python file in `src/shared/mapgen`

This document describes the current implementation in the repository, including every module, public function, internal helper, data structure, algorithm, input/output contract, and important edge case. It is source-oriented documentation rather than a general game design summary.

Generated from the Dungeon Quest 1 source tree

1. Package Overview

The mapgen package creates one complete dungeon world before a gameplay run begins. It represents a map as a rectangular list of integer tile IDs. The package first creates candidate grids, then applies a structured nine-sector layout, smooths local patterns, repairs disconnected areas, scores candidates against the player profile, and selects the best result. Finally, it places objectives, rewards, monsters, lights, and environmental decorations and wraps everything in a `GeneratedWorld` dataclass.

Module	Responsibility	Main dependency role
<code>constants.py</code>	Tile IDs, map defaults, asset roots	Shared vocabulary and configuration
<code>types.py</code>	<code>GeneratedWorld</code> runtime container	Output contract consumed by the client
<code>genetic_algorithm.py</code>	Seed, crossover, mutation	Candidate population creation
<code>layout.py</code>	Sector carving and corridors	Structured dungeon geometry
<code>cellular_automata.py</code>	Local neighborhood smoothing	Visual and spatial cleanup
<code>bfs.py</code>	Reachability, distances, repair, metrics	Playability guarantees
<code>astar.py</code>	Grid path existence checks	Placement validation
<code>decision_tree.py</code>	Profile scoring and recent rules	Adaptive selection and difficulty
<code>placement.py</code>	Objectives, items, monsters, decoration	Populate the final grid
<code>generator.py</code>	Pipeline coordinator	Public <code>generate_map</code> entry point
<code>__init__.py</code>	Package exports	Convenient import surface

The normal runtime entry point is `shared.mapgen.generate_map`. The client calls it with a profile and an optional seed. The generator owns the sequencing; the other modules are focused transformations and validators.

2. End-to-End Execution

For the default configuration, `generate_map` creates an 81 by 81 map, an initial population of eight grids, and four generations. Each generation processes every candidate through the same sequence: carve nine sectors, smooth the result, repair connectivity, and calculate metrics. Candidates from all generations are retained. The next population is bred from the two highest-scoring current candidates. After the final generation, `choose_best_candidate` selects from the accumulated candidates, not only the final population. Entity placement then uses the final reachable set and the profile-derived monster limit.

- `random.Random(seed)` makes all random choices in this pipeline reproducible when a stable seed is supplied.
- The layout stage reconstructs a wall-filled grid, so the genetic seed controls local openness and terrain influence rather than being copied directly to the output.
- BFS treats floors, water, treasure, scrolls, keys, and lava as walkable; only walls and traps are excluded from traversal.
- The final reachable set is sorted before placement so that placement receives stable input ordering even when the set itself is unordered.
- Scheduled monsters are separated from active monsters and are released later by `GeneratedWorld.update`.

The pipeline is deliberately explainable. There is no trained model: the genetic algorithm is search over candidate maps, BFS and A* are graph algorithms, and profile adaptation is a small rule system.

3. constants.py

File purpose: define the shared numeric tile vocabulary, default generation dimensions, sector count, and asset directories. No functions or classes are defined in this module.

Name	Value	Meaning
TILE_WALL	0	Solid wall; excluded from WALKABLE_TILES
TILE_TRAP	1	Trap tile; currently not traversable by mapgen BFS
TILE_FLOOR	2	Ordinary walkable floor
TILE_WATER	3	Walkable water with gameplay effects handled elsewhere
TILE_TREASURE	4	Walkable treasure location
TILE_BASIC_SCROLL	5	Walkable common consumable location
TILE_KEY	6	Walkable key location
TILE_RARE_SCROLL	7	Walkable rare consumable location
TILE_LAVA	8	Walkable lava with gameplay effects handled elsewhere
WALKABLE_TILES	2,3,4,5,6,7,8	Tile IDs accepted by reachability and pathfinding
SOLID_TILES	0	Declared solid tile set
SECTOR_COUNT	3	Creates a 3 by 3, nine-sector layout
DEFAULT_MAP_WIDTH/HEIGHT	81	Default grid dimensions
DEFAULT_POPULATION_SIZE	8	Candidate grids per generation
DEFAULT_GENERATIONS	4	Evolution rounds

TILES_ROOT is resolved relative to the package: it points to src/shared/tiles. ENVIRONMENT_ROOT, ITEM_ROOT, and MONSTER_ROOT are derived paths used by placement.py to build asset references. The constants module has no runtime side effects beyond constructing these Path objects.

4. types.py

Coordinate is a type alias for a pair of integers in row, column order. GeneratedWorld is the central output object. Its fields contain the grid and all objects needed by the renderer and gameplay screen.

Field	Type	Purpose
width, height	int	Grid dimensions
tiles	List[List[int]]	Mutable tile ID grid
player_spawn	Coordinate	Initial player row and column
door_key, door	Dict[str, object]	Progression objectives
treasures, keys, scrolls	list of dictionaries	Collectible and reward entities
monsters	list of dictionaries	Currently active monsters
wall_lights, decorations	list of dictionaries	Visual/environment objects
spawn_schedule	list of dictionaries	Delayed monster events
reachable_tiles	Set[Coordinate]	Walkable region from spawn
elapsed	float	Accumulated gameplay time, initially 0

GeneratedWorld.update(dt) advances elapsed time, sorts spawn_schedule by spawn_at, and moves every event whose time has arrived into monsters. It returns the monsters spawned during that call. Because events are popped from the schedule, each scheduled monster is emitted once. is_within_bounds is the bounds predicate; tile_at returns 0 for out-of-bounds coordinates; set_tile silently ignores out-of-bounds writes.

5. genetic_algorithm.py

This module supplies candidate grids and simple evolutionary operators. It does not evaluate candidates or decide which one wins; generator.py and decision_tree.py handle those responsibilities.

seed_layout(width, height, rng)

Creates a wall-filled grid and randomizes only interior cells. Each interior cell receives a terrain according to cumulative random thresholds: wall below 0.18, floor below 0.52, water below 0.62, trap below 0.75, basic scroll below 0.87, key below 0.95, and rare scroll otherwise. The outer border remains walls. The result is a list of independent row lists.

generate_population(width, height, population_size, rng)

Calls seed_layout population_size times using the same random generator and returns the resulting list of TileGrid values. It does not validate dimensions or remove duplicate candidates; duplicates are valid because later mutation and layout processing can still produce different outcomes.

crossover(parent_a, parent_b, rng)

Chooses one interior split row and one interior split column. It starts as a copy of parent_a, then copies every parent_b cell where row is at or below the split row or column is at or beyond the split column. This produces a union of large regions rather than a single horizontal or vertical cut. The function assumes compatible rectangular parents.

mutate(grid, rng, mutation_rate)

Copies the grid and considers only interior cells. A cell mutates when rng.random() is below mutation_rate. The replacement distribution is floor 55%, water 15%, trap 12%, wall 10%, and rare scroll 8%. Existing tile identity is not preserved, so mutation can remove keys or scrolls and can create terrain that later stages must make playable.

6. layout.py

layout.py converts a candidate seed into a structured nine-sector dungeon. It intentionally starts from all walls, carves local rooms and paths, connects sector centers, and then seeds sparse traps.

carve_nine_sector_maze(seed_grid, rng)

Creates a same-sized all-wall grid, protects the outer border, divides the grid into three rows and three columns, and calls carve_sector for all nine sectors. It then calls connect_sector_centers and seed_traps. The seed grid is read by carve_sector to bias which cells open, but the output is newly constructed.

carve_sector(grid, seed_grid, sector_row, sector_col, sector_h, sector_w, rng)

Begins at the sector center and performs max(40, sector area divided by two) random-walk steps. At each step it examines the surrounding 3 by 3 neighborhood. Seed floor or water cells, and 24% of other cells, are opened; 10% of opened cells become water. The walker is then moved one cardinal step while clamped inside the sector. There is a 30% chance of carving an additional 5 by 5 center patch. Sector boundaries and the global border are preserved by bounds checks.

connect_sector_centers(grid, sector_h, sector_w, rng)

Builds sector centers and links them in a serpentine path through all nine sectors, guaranteeing a designed connection chain. It also adds optional horizontal and vertical links, each with probability 0.55. Each link is carved by carve_corridor.

carve_corridor(grid, start, end, rng) and carve_band(grid, row, col)

carve_corridor randomly chooses whether to travel along columns or rows first, walks one cell at a time, and calls carve_band. carve_band opens a bounded 3 by 3 square around each route coordinate but never opens the outer border.

seed_traps(grid, sector_h, sector_w, rng) and has_trap_neighbor(grid, row, col, radius)

seed_traps examines interior cells within each sector. A floor cell has a 3% chance to become a trap if no trap exists within Manhattan or square-window radius 2 as implemented by has_trap_neighbor. The spacing check prevents dense adjacent trap clusters. has_trap_neighbor scans a bounded rectangular window and excludes the center cell.

7. cellular_automata.py

`smooth(grid, steps=2)` applies a synchronous local-neighborhood transformation. It copies the input first, so callers do not receive an in-place mutation. For each interior cell, it counts wall and trap neighbors across the eight surrounding cells using the previous iteration's grid.

- Five or more wall neighbors force the cell to `TILE_WALL`.
- A water cell with at most two wall neighbors becomes ordinary floor.
- A trap cell with at most one wall neighbor becomes ordinary floor.
- All other cells keep their previous value.
- Each completed iteration replaces current with `next_grid`, so changes do not cascade within the same pass.

Only interior cells are changed; the border is preserved. The function does not repair connectivity, and it does not consider walkability directly. That is why `generator.py` runs BFS repair after smoothing.

8. bfs.py

`bfs.py` contains the reachability model used for spawn selection, metrics, farthest objectives, and connectivity repair. Every traversal uses four-directional neighbors and `WALKABLE_TILES`.

`find_spawn_tile(grid)`

Scans interior cells that are walkable and calculates Manhattan distance from the grid center. After sorting, it returns the walkable cell nearest to the center. If no candidate exists, it returns `(max(1, center_row), max(1, center_col))`; this fallback is a coordinate, not a guarantee that the fallback tile is walkable.

`reachable_tiles(grid, start)`

Performs breadth-first search with a deque. It enqueues neighbors even before bounds and walkability checks; invalid entries are discarded when popped. The returned set includes the start only if it is in bounds and walkable. This set is the authoritative connected region.

`distance_map(grid, start)`

Uses BFS to return a coordinate-to-distance dictionary. The first distance assigned is shortest because the queue is FIFO. Invalid or blocked cells are omitted. Distances are measured in four-directional steps.

`farthest_reachable_tile(grid, start, exclude=None)`

Builds a distance map, removes excluded coordinates from consideration, and returns the maximum distance. Ties are resolved by the tuple `(distance, -row, -column)`, which makes the choice deterministic. If the distance map is empty or all coordinates are excluded, it returns start.

`evaluate(grid, start=None)`

Calculates `start_row`, `start_col`, `reachable_count`, `walkable_count`, `reachable_ratio`, and `open_ratio`. `reachable_ratio` is `reachable_count` divided by `walkable_count`, while `open_ratio` is `walkable_count` divided by total grid area. Both denominators use `max(1, denominator)` to avoid division by zero. The returned dictionary is consumed by candidate scoring.

`repair_connectivity(grid, start=None)` and `_carve_path(grid, start, end)`

`repair_connectivity` copies the grid, identifies all walkable cells outside the start region, orders them by Manhattan distance from start, and connects each target to its nearest currently reachable cell. `_carve_path` moves row-first or column-first toward the target and writes `TILE_FLOOR` along the route. After every target, reachability is recomputed. This guarantees that previously isolated walkable regions become connected, although it can overwrite terrain with floor.

9. astar.py

astar.py provides path validation. It uses four-directional movement, unit edge costs, and the Manhattan heuristic. Both functions accept a rectangular grid, a start coordinate, and a goal coordinate.

find_path(grid, start, goal)

Returns a list of coordinates from start to goal, inclusive. It uses a heap ordered by $f = g + h$ and a monotonically increasing sequence number to make heap entries comparable when f values tie. `came_from` reconstructs the path after the goal is popped. The start-equals-goal case returns `[start]`. If no route exists, it returns an empty list.

path_exists(grid, start, goal)

Returns a boolean and avoids storing a path. It maintains g scores, a closed set, and a list of f -score entries that is sorted before each pop. This is functionally A* but less efficient than heap-based `find_path` because sorting the open list is repeated. It is used by monster placement to reject unreachable spawn positions.

Important contract: astar.py reads `WALKABLE_TILES` from `constants.py`, so traps are not traversable even though the gameplay layer may display or interact with them. The map generator's connectivity guarantees and A* placement check therefore use the same walkability definition.

10. decision_tree.py

Despite its filename, this module does not implement a conventional CART decision tree. It contains profile-aware scoring plus a small association-rule calculation used for adaptive difficulty.

mine_recent_rules(profile)

Reads at most the last five dictionaries from `profile['last_five_games']`. Each valid game becomes a transaction containing `outcome:clear` or `outcome:loss`, `performance:high` when `monsters_killed` is at least 8 or `performance:low` otherwise, and `item:<name>` entries for collected items. It then measures two manually defined rules: high performance to clear outcome and low performance to loss outcome. The returned value is confidence, calculated as matching transactions divided by antecedent transactions. Items are included in transactions but are not currently used as antecedents or consequents.

monster_limit_for_profile(profile)

Returns 12 for an empty or invalid history. It counts the consecutive losses at the end of the last five games. Two or more losses plus `low_kill_loss` confidence of at least 0.5 produces a multiplier of 1.35 plus up to 0.20 for longer streaks. Otherwise, `high_kill_clear` confidence of at least 0.5 produces a multiplier of 1.15. The result is rounded and clamped from 1 through 18.

score_candidate(metrics, profile)

Scores a map using profile level, total historical monster kills, total item usage, open ratio, reachable ratio, and reachable count. Levels 1-2 target `open_ratio` 0.42, levels 3-5 target 0.52, and levels above 5 target 0.62. Reachability is always strongly rewarded; the contribution of map size or historical activity changes by level.

choose_best_candidate(candidates, profile)

Returns the candidate with the highest `score_candidate` result. `reachable_ratio` is the tie-breaker. Each candidate must provide a metrics dictionary with the expected keys and a grid, although only metrics are read by the selector.

11. placement.py

placement.py turns a connected tile grid into game content. It produces plain dictionaries because the client and renderer consume dictionary-shaped entity data. Asset references are generated as POSIX-style paths relative to `src/shared/tiles`.

place_entities(grid, spawn, reachable, rng, monster_limit=12)

Maintains an excluded coordinate set beginning with spawn. It chooses the door key and door using `farthest_reachable_tile`, then places treasures, their keys, scrolls, monsters, wall lights, and environmental decorations in that order. Every placement expands excluded, preventing collisions. The returned dictionary contains `door_key`, `door`, `treasures`, `keys`, `scrolls`, `monsters`, `wall_lights`, and `decorations`.

place_treasures(...)

Shuffles available reachable cells, chooses two or three locations, changes those grid cells to `TILE_TREASURE`, and assigns each either a Bow Gun or Light reward. Each treasure starts closed and requires a generic key. Images and reward metadata are included in the dictionary.

place_keys(...)

Creates one key for each treasure. For each target treasure it chooses the available coordinate with greatest Manhattan distance from that treasure, changes the tile to `TILE_KEY`, and records the key. The door-key coordinate is restored to `TILE_FLOOR` after keys are placed because the door key is represented separately in the returned `door_key` object.

place Scrolls(...)

Randomly chooses up to six available cells. The first four use `TILE_BASIC_SCROLL` and the last two use `TILE_RARE_SCROLL`. Each receives a random potion-like kind and image metadata. The function does not guarantee one of each potion type.

place_monsters(...)

First places guarding monsters: one monster3 at a random choice and one alternating monster type near each treasure. It then fills remaining capacity with non-guarding monsters, selecting only coordinates with an A* path back to spawn. Non-guarding monsters receive `spawn_at` values beginning at 2.5 seconds and increasing by 2.5 seconds per candidate index. Guarding monsters have no `spawn_at` and are active immediately. The monster list is capped by `monster_limit`, but guards are added before that cap is checked, so the guard set can exceed a small limit.

place_wall_lights(...)

Scans interior wall cells and places a wall light with 12% probability when the cell below is not a wall. It does not add coordinates to excluded because lights occupy wall positions rather than walkable entity slots.

place_environment_decorations(...) and place_liquid_cluster(...)

Creates one lava fountain and one water fountain cluster, then two smaller lava and water groups. A cluster chooses random available centers, optionally adds a fountain image, and converts cells within a Manhattan radius to the selected liquid tile while recording decoration dictionaries. `excluded` prevents clusters and important content from overlapping.

nearest_free_position(...), monster_frames(...), rel_path(...)

`nearest_free_position` finds the closest non-excluded choice within a radius. `monster_frames` builds four animation paths for a monster name. `rel_path` resolves an asset path from the shared tiles root and converts separators to forward slashes.

12. generator.py

generator.py is the orchestrator and the public construction API. It imports every major stage and exposes `generate_map`, `score_key`, and `breed_next_population`.

`generate_map(width=..., height=..., profile=None, seed=None)`

The function creates a local `random.Random`, normalizes a missing profile to an empty dictionary, and generates the initial population. For each generation it transforms every grid, evaluates metrics, sorts the current population with `score_key`, and breeds the next population. All processed candidates are accumulated. It then selects the best candidate with `choose_best_candidate`, calculates spawn and reachability, calls `place_entities` with `monster_limit_for_profile`, separates immediate monsters from scheduled monsters, and returns `GeneratedWorld`.

`score_key(metrics, profile)`

This is the evolutionary-stage scoring function. It uses the same profile activity signals and level bands as `score_candidate`, but it is kept locally in `generator.py`. As a result, generation ranking and final selection use equivalent but duplicated scoring formulas. The function rewards reachability and proximity to the level-specific openness target, with historical kills and item use contributing at mid and high levels.

`breed_next_population(scored_population, rng, generation)`

Takes the first two already-sorted candidates as survivors. If only one exists it duplicates it. The first two children are the survivor grids themselves; subsequent children are created by random parent sampling, crossover, and mutation. Mutation rate is $0.06 + \text{generation} * 0.02$, giving 0.06, 0.08, 0.10, and 0.12 across the default four generations. The function assumes at least one scored candidate.

13. `__init__.py`

The package initializer re-exports all names from `constants.py`, imports `generate_map` from `generator.py`, and imports `GeneratedWorld` from `types.py`. This lets callers use `shared.mapgen.generate_map` and `shared.mapgen.GeneratedWorld` without importing the implementation modules directly. The wildcard constants export also exposes tile IDs and generation defaults at package level.

14. Data Flow and Contracts

Stage	Input	Output	Invariant
Seed	dimensions, RNG	candidate grids	border is wall
Layout	seed grid, RNG	structured grid	border stays wall; centers are connected
Smoothing	grid, steps	copied grid	input is not mutated
BFS repair	grid, optional start	copied connected grid	reachable walkable regions are joined
Evaluation	grid, optional start	metrics dictionary	ratios avoid zero division
Selection	candidate metrics, profile	best candidate	reachable ratio breaks ties
Placement	grid, spawn, reachable, RNG	entity dictionaries	excluded coordinates avoid most collisions
World wrapping	grid and entities	<code>GeneratedWorld</code>	scheduled events release once

The `mapgen` package does not load image files, create Pygame sprites, or move monsters. It only returns tile IDs and metadata. Rendering and real-time behavior belong to the client layer. This separation is important when debugging: a correct `mapgen` dictionary can still be rendered incorrectly by a consumer, and a rendering issue does not necessarily indicate a generation failure.

15. Complexity and Edge Cases

- Grid operations are proportional to map area for each pass. With eight 81 by 81 candidates and four generations, layout, smoothing, and metric work dominate normal generation time.
- `reachable_tiles` and `distance_map` are $O(V + E)$ for the walkable grid; with four neighbors, this is linear in the number of cells.
- `repair_connectivity` can repeat BFS for every disconnected target, making it more expensive than a single traversal on highly fragmented input.
- `find_path` uses a heap, while `path_exists` repeatedly sorts a list. The latter is adequate for placement-scale checks but is not the faster implementation.
- Empty grids and empty rows are guarded in several functions, but callers normally provide rectangular non-empty grids.
- The fallback spawn coordinate can point to a blocked tile when a grid has no walkable interior. Normal generation avoids this through carving and repair, but the fallback is still worth testing.
- Placement assumes enough reachable choices for some content. It generally skips an item when no position exists, so small custom maps may contain fewer entities than defaults.
- The package treats water and lava as walkable for graph algorithms. Movement speed, damage, or effects are applied by gameplay code, not by mapgen.

16. Practical Debugging Checklist

- Call `generate_map(seed=known_value)` twice and compare tiles and entity dictionaries when investigating reproducibility.
- Check `world.player_spawn` is in `world.reachable_tiles` and that `tile_at(spawn)` is in `WALKABLE_TILES`.
- Confirm `door_key` and door coordinates are distinct and do not collide with spawn or treasure/key coordinates.
- Check every non-guarding monster with `astar.path_exists` back to spawn; placement performs this check before insertion.
- Inspect `spawn_schedule` and call `update` with controlled `dt` values to verify delayed monster release.
- Use `evaluate` to compare `walkable_count`, `reachable_count`, `reachable_ratio`, and `open_ratio` before and after each transformation.
- When a profile changes difficulty unexpectedly, inspect `last_five_games`, `monsters_killed` thresholds, final loss streak, and the two confidence values returned by `mine_recent_rules`.

Conclusion: mapgen is a complete procedural content pipeline with clear module boundaries. The key architectural contract is that `generator.py` produces a connected, populated `GeneratedWorld` while algorithms remain independent and reusable. The strongest correctness checks are deterministic seeded generation, connectivity, valid objective placement, path-valid monsters, and scheduled-spawn behavior.